

Here's the EPIC EHR OAuth flow—clean, step-by-step—covering both **interactive SMART on FHIR (Authorization Code + PKCE)** and **server-to-server (SMART Backend Services)**. I'll use EPIC terms where it matters and point out EPIC-specific quirks.

0) One-time setup (both flows)

1. Register your app (EPIC developer portal)

- Get a **client_id** and register **exact** redirect URL(s).
- Choose audiences (patient, provider, backend system) and request the scopes you need (e.g., `patient/Encounter.read`, `system/Encounter.read`, `openid fhirUser offline_access`).
- For backend, upload a **JWK set** (RSA public keys) EPIC will use to verify your JWT. ([Epic on FHIR](#), docs.intraconnects.com)

2. Know your FHIR base ("iss") per site

- Each EPIC customer (hospital/health system) has its own **FHIR base URL** (= `iss`). Your tokens are valid for that issuer only. ([Epic on FHIR](#))

3. Discover endpoints at runtime

- Fetch `/.well-known/smart-configuration` from the `iss` to learn the **authorize**, **token**, **revocation**, **introspection**, **JWKS**, supported **PKCE** methods, etc.
 - Example (EPIC R4 sandbox): shows `code_challenge_methods_supported: ["S256"]`. ([Epic on FHIR](#))
 - This well-known discovery is a SMART requirement; EPIC also exposes OpenID discovery at `/.well-known/openid-configuration`. ([FHIR Builder](#), [Epic on FHIR](#))
-

A) Interactive apps (Authorization Code + PKCE) – EHR Launch & Standalone

Use this for clinician- or patient-facing apps.

Why PKCE? SMART requires clients to support PKCE, and EPIC supports **S256 only** (not `plain`). ([FHIR Builder](#), [Epic on FHIR](#))

A1) EHR Launch (from inside EPIC)

1. EPIC launches your app with query params like:

`launch=<token>&iss=<FHIR base>`

Your requested scopes must include `launch` plus context scopes like `launch/patient` and/or `launch/encounter` if you need those IDs automatically. ([FHIR Builder](#))

2. Build the authorization URL (from well-known):

- `response_type=code`
- `client_id=...`
- `redirect_uri=...`
- `scope=openid fhirUser launch launch/patient patient/Encounter.read offline_access` (example)
- `state=<random>`
- `aud=<FHIR base>` (often required; in EHR launch, `aud` equals `iss`)
- **PKCE:** `code_challenge=<S256(verifier)>` , `code_challenge_method=S256`
- Include the `launch=<token>` you received. ([FHIR Builder](#))

3. User signs in & consents in EPIC; EPIC redirects you to your `redirect_uri` with `code` and `state` .([FHIR Builder](#))

4. Exchange code for tokens (POST to token endpoint):

- `grant_type=authorization_code` , `code=...` , `redirect_uri=...` , `client_id=...` , `code_verifier=...` .
- Response: `access_token` , `expires_in` , `scope` , and `refresh_token` if you requested `offline_access` .([FHIR Builder](#))

5. Call FHIR using `Authorization: Bearer <access_token>` at the `iss` base. (Example GETs below.)

A2) Standalone Launch (outside EPIC)

Same as above **without** the `launch` parameter. Many sites require `aud=<FHIR base>` even for standalone launches. (docs.intraconnects.com)

Minimal cURL examples (interactive flow):

```
# 1) Discover endpoints (replace <iss>)
curl -sS "<iss>/.well-known/smart-configuration" | jq .
```

```
# 2) After redirect back with ?code=...
# Exchange code for tokens
curl -X POST "<token_endpoint>" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=authorization_code" \
  -d "code=<auth_code>" \
  -d "redirect_uri=<your_redirect_uri>" \
  -d "client_id=<your_client_id>" \
  -d "code_verifier=<your_pkce_verifier>"
```

SMART requires this sequence and EPIC's discovery documents provide the URLs and PKCE details.
([FHIR Builder](#), [Epic on FHIR](#))

B) Server-to-server (SMART Backend Services) – “backend service for encounters”

Use this when **no user** is present (e.g., nightly jobs, integrations). You'll request **system-level scopes** like `system/Encounter.read`.

B1) Create and sign a short-lived JWT

- Claims:
 - `iss` : your client_id
 - `sub` : your client_id
 - `aud` : **token endpoint URL** from discovery
 - `exp` : now + 5 minutes (typ.)
 - `jti` : random UUID
- Sign with your **RS256 private key** that matches the JWK you uploaded to EPIC. ([FHIR Builder](#), [docs.intraconnects.com](#))

B2) Exchange JWT for access token (client_credentials)

POST to token endpoint:

```
curl -X POST "<token_endpoint>" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=client_credentials" \
  -d "scope=system/Encounter.read" \
  -d "client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt" \
  -d "client_assertion=<your_signed_JWT>"
```

Response includes `access_token` and `expires_in`. (This is the SMART Backend Services profile, which EPIC implements for backend apps.)([FHIR Builder](#), [docs.intraconnects.com](#))

B3) Call FHIR Encounter

Examples(R4):

```
# Encounters updated recently (site-specific filters vary)
curl -H "Authorization: Bearer $TOKEN" \
  "<iss>/Encounter?_lastUpdated=ge2025-08-01T00:00:00Z"
```

```
# By patient
curl -H "Authorization: Bearer $TOKEN" \
  "<iss>/Encounter?patient=<patient_id>"
```

```
# Read by ID
curl -H "Authorization: Bearer $TOKEN" \
  "<iss>/Encounter/<encounter_id>"
```

Use the **issuer's** FHIR base (`iss`) and your **system scope**. Actual access depends on what the EPIC site approved for your app. ([Epic on FHIR](#))

Common gotchas (EPIC-specific tips)

- **PKCE = S256 only**; no `plain`. ([Epic on FHIR](#))
- **Exact redirect URI match** is enforced; register all that you'll use. (SMART standard.)([FHIR Builder](#))
- `aud` is often required; for EHR launches it equals the `iss`. ([FHIR Builder](#))
- **Discovery first**: always read `/.well-known/smart-configuration` for endpoints/features. ([FHIR Builder](#))
- **Per-site enablement**: even with a sandbox/"global" `client_id`, production access is **per EPIC customer** and credentials/tokens should be **unique per org**. Don't expect one token to work across multiple orgs. ([Epic on FHIR](#))

Quick chooser: which flow?

- **Clinician or patient uses your UI?** → **Authorization Code + PKCE** (EHR Launch or Standalone). ([FHIR Builder](#))
 - **Headless integration / nightly jobs / system-to-system?** → **SMART Backend Services** (`client_credentials` with JWT). ([FHIR Builder](#))
-

If you want, I can sketch a minimal Rails controller for each token exchange and an Encounter fetch using the EPIC R4 sandbox issuer and scopes you plan to use.